Step 6.4: Polygon Classification and Relationship Analysis

Comprehensive Implementation Summary

V6_current.py

December 18, 2025 – Version 1.1 (With Corrections)

1 Overview

Step 6.4 is a **polygon classification and relationship analysis system** that processes polygons within each face to determine their roles and relationships. It runs at **line 3723** and is part of the face construction pipeline.

2 Current Implementation Breakdown

2.1 1. Visibility-Based Classification Method

Location: Lines 3725–3950

2.1.1 Method Details

- Uses **Shapely geometric analysis** with boundary polygon as reference
- Computes intersection areas between each polygon and the boundary
- Analyzes edge sharing patterns between polygons
- Checks if polygon edges cross through boundary interior

2.1.2 Classification Logic

```
if intersection_area > 1e-6 and shared_edges > 0:
    if poly_edges_cross_boundary:
        -> REPLACES BOUNDARY (boundary is invalid)
    else:
        -> ARTIFACT (delete - overlaps + shares edges)

elif boundary.contains(poly) and shared_edges == 0:
    -> HOLE (contained within boundary, no edge sharing)

elif intersection_area < 1e-6:
    if shared_edges > 0:
        -> ALTERNATE (touches at edges, zero intersection)
    else:
        -> SEPARATE FACE (completely independent)
```

2.1.3 Output Categories

- `BOUNDARY`: Main face boundary (1 per face)
- `ALT`: Alternative boundary polygons
- `HOLE`: Interior holes
- `SEPARATE FACE`: Independent polygons → new faces
- `ARTIFACT`: Invalid overlaps → deleted

2.2 2. Independent Boundary Detection

Location: Lines 5200–5450

2.2.1 Process

1. Identifies polygons marked as "independent boundaries" from earlier processing
2. Groups them by **connectivity** (touching/intersecting polygons)
3. Creates **one new face per connectivity group**
4. Largest polygon in group → `BOUNDARY`
5. Others in group → `ALT`
6. Holes assigned to parent polygon in same group

Output: New faces added to `new_faces_to_add[]`

2.3 3. Deduplication Scope

2.3.1 Current Reality

- **Within-face deduplication:** Yes – done during classification (artifacts removed)
- **Cross-face deduplication:** **NO** – not explicitly implemented
- **Gap:** Identical polygons from different faces can coexist

Method: Artifact detection based on intersection + edge sharing (not true polygon identity matching)

2.4 4. Initial Edge Distribution Analysis

Location: Lines 5850–5870

2.4.1 Analysis

```
# Builds edge_face_map_all from ALL polygons
for edge:
    count faces sharing this edge

Classifies:
- Boundary edges: 1 face (problematic - open edge)
- Manifold edges: 2 faces (good - closed mesh)
- Invalid edges: 3+ faces (bad - topology error)
```

Triggers extraction if: `invalid_edges > 0`

2.5 5. Initial Pruning Logic

Location: Lines 5970–6590

2.5.1 Method Summary

```
LOOP (up to 5 iterations or until <20 polygons):
    1. Identify polygons contributing to invalid edges
    2. Group into "units" (BOUNDARY+holes, ALT+holes move together)
    3. For each unit:
       - Calculate: invalid_shared (edges causing problems)
                    invalid_unique (edges not in other units)
       - Score = invalid_shared + 0.5 * invalid_unique
    4. Rank units by score (highest = worst offender)
    5. Test removal: If invalid edges drop, return unit to base
    6. Special case: If ALL units clean (0 invalid), remove cleanest
    7. Rebuild edge maps and repeat

EXIT WHEN:
- No invalid edges remain
- <20 polygons in extraction set
- 5 iterations reached
```

2.5.2 Output

- `invalid_edge_polygons[]` – extracted set for combination testing
- `pruned_polygons[]` – removed entirely
- `returned_to_base[]` – moved back to base set

2.6 6. Build Extraction List and Base Set

Location: Lines 5870–5970

2.6.1 Process

1. Find all polygons with edges in 3+ faces
2. Group them into units (BOUNDARY+holes, each ALT+its holes)

3. Create `invalid_edge_polygons[]` (extraction set)
4. Remaining polygons stay in base set
5. Build `polygon_units[]` for group-based operations

2.6.2 Data Structure

```
poly_info = {
    'face_idx': int,
    'polygon_idx': int,
    'data': polygon dict with 'vertices',
    'poly_type': 'BOUNDARY'|'ALT'|'HOLE',
    'face_eq': reference to unique_faces entry
}
```

2.7 7. Face Representation Analysis

Location: Lines 6650–6730

2.7.1 Current Implementation

```
# Categorize faces based on base set representation
faces_with_base = {}       # Faces already in base -> OPTIONAL
faces_without_base = {}    # Faces missing from base -> REQUIRED
mandatory_polygons = []    # Single-polygon faces -> MUST include

# For faces with only 1 polygon in extraction: auto-add to mandatory
```

2.7.2 Implemented Features

[UPDATED]

Hole-to-boundary dependency checking [NEW]

- When a HOLE polygon is mandatory, automatically includes its BOUNDARY
- Searches for BOUNDARY in same face as the HOLE
- Prevents incomplete face representations

Invalid edge verification [NEW]

- Before adding single-polygon faces, tests if they create invalid edges
- Computes edge statistics with test polygon added to base
- Skips polygons that would introduce topology errors
- Moves problematic single-polygon faces to multi-polygon handling

2.8 8. Single-Polygon Face Handling

Location: Lines 6680–6710

2.8.1 Current Logic

[UPDATED]

```
if face has only 1 polygon in extraction set:
    # Test if adding it creates invalid edges
    test_edge_map = base_edge_map + polygon_edges
    if test_invalid == 0:
        -> Add to mandatory_polygons[]
    else:
        -> Move to multi-polygon handling (remaining_faces_no_base)
```

IMPLEMENTED: Invalid edge verification before returning (see Section 2.7)

2.9 9. Combinations for No-Base-Representation Faces

Location: Lines 6730–6840

2.9.1 Current Method

[UPDATED]

```
# STEP 2: Handle faces without base representation
# Collect all polygons from faces without base
all_no_base_polygons = []
for face_idx in remaining_faces_no_base:
    all_no_base_polygons.extend(remaining_faces_no_base[face_idx])

# Test subsets from LARGEST to SMALLEST
max_combinations = 4000  # Increased from 100
for subset_size in range(len(all_no_base_polygons), 0, -1):
    for combo in combinations(all_no_base_polygons, subset_size):
        test_combo = mandatory_polygons + combo
        # Evaluate: invalid edges (priority), boundary edges (secondary)
        if test_invalid < best_invalid:
            best_combination = test_combo

        # Early exit if: invalid=0 and boundary=0
        if test_invalid == 0 and test_boundary == 0:
            break

    # If found perfect solution, don't test smaller subsets
    if perfect_solution_found:
        break
```

Key Changes:

- **Strategy:** Changed from `product()` (one polygon per face) to `combinations()` (subsets of all polygons)
- **Direction:** Tests from most polygons to fewest (greedy approach)
- **Limit:** Increased from 100 to 4000 combinations
- **Optimization:** Stops testing smaller subsets once perfect solution found

Optimization: Caches base edge map to avoid rebuilding

2.10 10. Edge Map Rebuilding

Location: Multiple places (6950–7030, 6600–6650)

2.10.1 When

- After pruning iterations
- After augmentation (combination selection)
- After post-augmentation extraction (NEW feature)

2.10.2 Process

```
# Clear and rebuild from scratch
edge_face_map_all = {}
for each face:
    for each polygon (if not removed):
        register all edges with (face_idx, poly_idx, poly_type)

# Compute statistics: boundary, manifold, invalid counts
```

2.11 11. Post-Augmentation Extraction

Location: Lines 6960–7036 [JUST IMPLEMENTED]

2.11.1 Trigger

If invalid edges > 0 after augmentation

2.11.2 Process

1. Find all edges in 3+ faces (invalid edges)
2. Identify all base polygons sharing these edges
3. Create `poly_info` dicts with proper structure
4. Move to extraction set, mark as `'EXTRACTED_DUE_TO_INVALID_EDGES_AFTER_AUGMENTATION'`
5. Rebuild edge map without these polygons
6. Continue to optimal combination search

Result: Seed 116 test – invalid edges $3 \rightarrow 0$, 5 polygons extracted

2.12 12. Optimal Combination Finding

Location: Lines 6840–6900

2.12.1 STEP 3

If still have invalid edges after STEP 2

2.12.2 Method

```
# Try adding polygons from faces with base representation (optional faces)
max_tests = min(50, 2^num_optional_faces)

for each combination of optional polygons:
    add to current best combination
    rebuild edge map
    evaluate statistics
    if invalid < best_invalid:
        update best combination

    if invalid == 0:
        -> Early exit
```

Limit: Max 20 polygons total in final combination (inherited from earlier logic)

2.13 13. ALT Polygon Testing (Removal/Merging)

Location: Lines 8100–8516 (after this summary)

2.13.1 Process

1. **Removal Test:** Try removing each ALT, check if face still valid
2. **Merge Test:** For ALTs sharing edges, try merging
3. **Output:** After ALT removal completion (line ∼8516)

3 Key Insights

3.1 Complexity Sources

1. **Multi-stage classification** (visibility → pruning → combination)
2. **Iterative pruning** (up to 5 loops with complex scoring)
3. **Combinatorial explosion** handling (100 combo limit, 50 optional tests)
4. **Group-based operations** (units must move together)
5. **Multiple edge map rebuilds** (after each modification)

3.2 Simplification Opportunities

- Reduce pruning iterations (currently 5 max)
- Simplify unit scoring (currently: shared + 0.5× unique)
- Merge STEP 2 & 3 (no-base vs optional faces)
- Add early validation to avoid later re-extraction

4 Validation Status

Visibility-based classification

Independent boundary detection

× Cross-face deduplication (not implemented)

Initial edge distribution analysis

Initial pruning logic

Build extraction list and base set

Hole-to-boundary dependency [IMPLEMENTED v1.1]

Invalid edge verification for single-polygon faces [IMPLEMENTED v1.1]

Combinations for no-base-representation [IMPROVED v1.1]

Edge map rebuilding

Post-augmentation extraction

Optimal combination finding

ALT polygon testing

5 Version 1.1 Changes (December 18, 2025)

5.1 Implemented Corrections

1. **Hole-to-Boundary Dependency (Step 7)**
   - Automatically includes BOUNDARY when HOLE is mandatory
   - Prevents incomplete face representations
   - Searches for parent BOUNDARY in same face
2. **Invalid Edge Verification (Step 7)**
   - Tests single-polygon faces before adding to mandatory set
   - Computes edge statistics with test polygon
   - Skips polygons creating invalid edges
   - Moves problematic cases to multi-polygon handling
3. **Improved Combination Strategy (Step 9)**
   - Changed from `product()` to `combinations()`
   - Tests subsets from largest (all polygons) to smallest
   - Increased limit from 100 to 4000 combinations
   - Early termination when perfect solution found

5.2 Rationale

- **Dependency checking:** Ensures face completeness, prevents topology errors
- **Edge verification:** Validates before commitment, reduces post-augmentation extraction
- **Subset testing:** More comprehensive search space, prioritizes including more polygons
- **Limit increase:** Allows thorough exploration while maintaining performance